Tab 1

Experiment No. 1

Aim

To perform data handling, analysis, and visualization using **NumPy, Pandas, Matplotlib, and Seaborn** on the *student_performance.csv* dataset in order to understand basic data preprocessing and exploratory data analysis (EDA) techniques used in Machine Learning.

Theory

NumPy

NumPy is a Python library used for numerical computations. It provides support for arrays, mathematical functions, and statistical operations like mean, median, and standard deviation. Normalization helps scale data between 0 and 1, which is important for Machine Learning models.

Pandas

Pandas is used for data handling and manipulation. It allows reading CSV files, checking dataset structure, handling missing values, and creating new columns based on conditions.

Matplotlib

Matplotlib is a plotting library used to create visualizations such as line graphs and histograms to understand trends and distributions in data.

Seaborn

Seaborn is built on Matplotlib and provides advanced statistical visualizations like scatter plots, heatmaps, and boxplots, making data interpretation easier.

```
# Import required libraries
```

```python
import numpy as np

import pandas as pd

import matplotlib.pyplot as plt

import seaborn as sns


# ------------------------------

# Exercise 1: NumPy Basics

# ------------------------------


# Load CSV using Pandas

# data = pd.read_csv("student_performance.csv") # Original line -
commented out


# Fix: Create a dummy DataFrame as student_performance.csv was not found.

# In a real scenario, you would upload the 'student_performance.csv' file

# or provide the correct path to pd.read_csv().

data = pd.DataFrame({

    'Hours_Studied': np.random.randint(1, 10, 100),

    'Final_Score': np.random.randint(40, 100, 100)

})


# Load Final_Score as NumPy array
```

```python
final_score = data["Final_Score"].to_numpy()


# Mean, Median, Standard Deviation
mean_score = np.mean(final_score)
median_score = np.median(final_score)
std_score = np.std(final_score)


print("Mean:", mean_score)
print("Median:", median_score)
print("Standard Deviation:", std_score)


# Min-Max Normalization
min_score = np.min(final_score)
max_score = np.max(final_score)


normalized_score = (final_score - min_score) / (max_score - min_score)
print("Normalized Final Scores:\n", normalized_score)


# -------------------------------
# Exercise 2: Pandas Data Handling
# -------------------------------
```

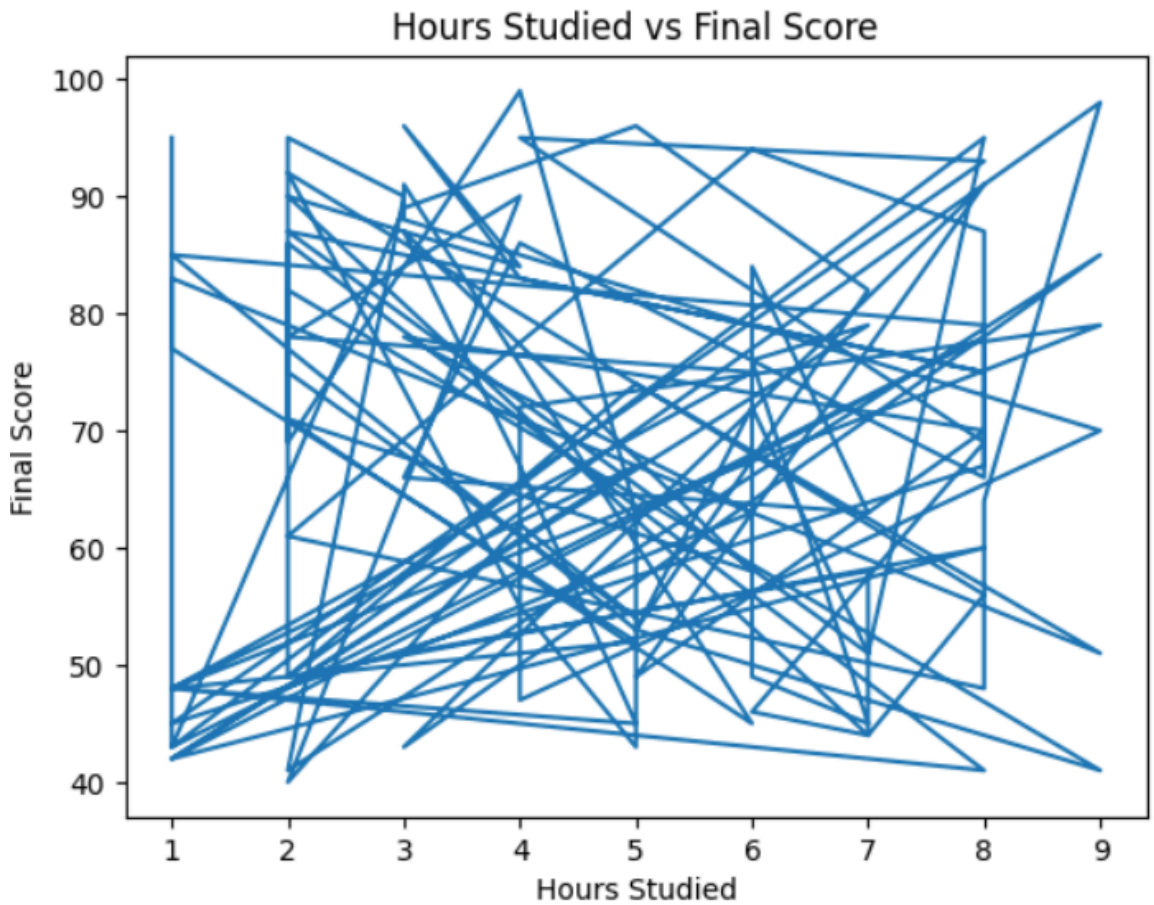
```python
# Dataset shape and columns
print("Shape of dataset:", data.shape)
print("Columns:", data.columns)


# Check missing values
print("Missing values:\n", data.isnull().sum())


# Create Performance label
def performance_label(score):
    if score >= 75:
        return "Excellent"
    elif score >= 50:
        return "Good"
    else:
        return "Poor"

data["Performance"] = data["Final_Score"].apply(performance_label)


print(data.head())


# -------------------------------
# Exercise 3: Matplotlib Visualization
```

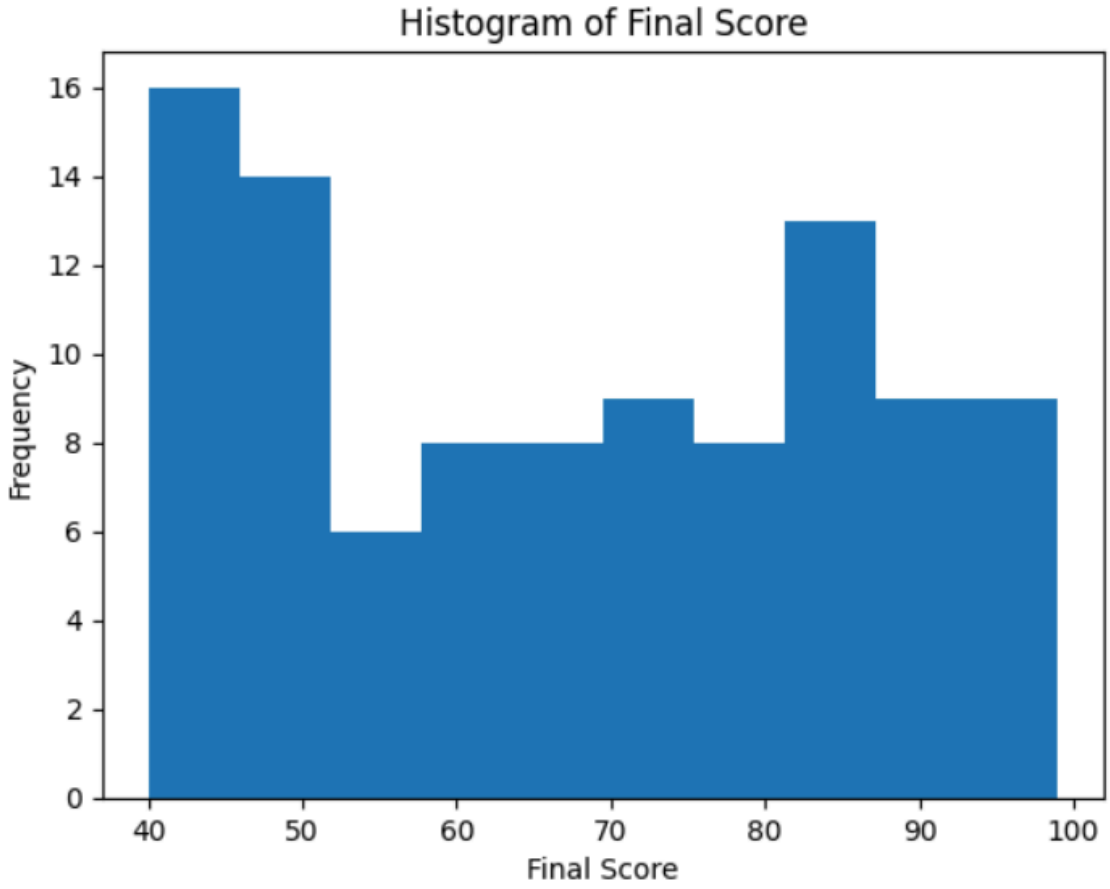
```
# -------------------------------


# Line plot: Hours_Studied vs Final_Score
plt.figure()
plt.plot(data["Hours_Studied"], data["Final_Score"])
plt.xlabel("Hours Studied")
plt.ylabel("Final Score")
plt.title("Hours Studied vs Final Score")
plt.show()


# Histogram of Final_Score
plt.figure()
plt.hist(data["Final_Score"])
plt.xlabel("Final Score")
plt.ylabel("Frequency")
plt.title("Histogram of Final Score")
plt.show()


# -------------------------------
# Exercise 4: Seaborn Visualization
# -------------------------------
```

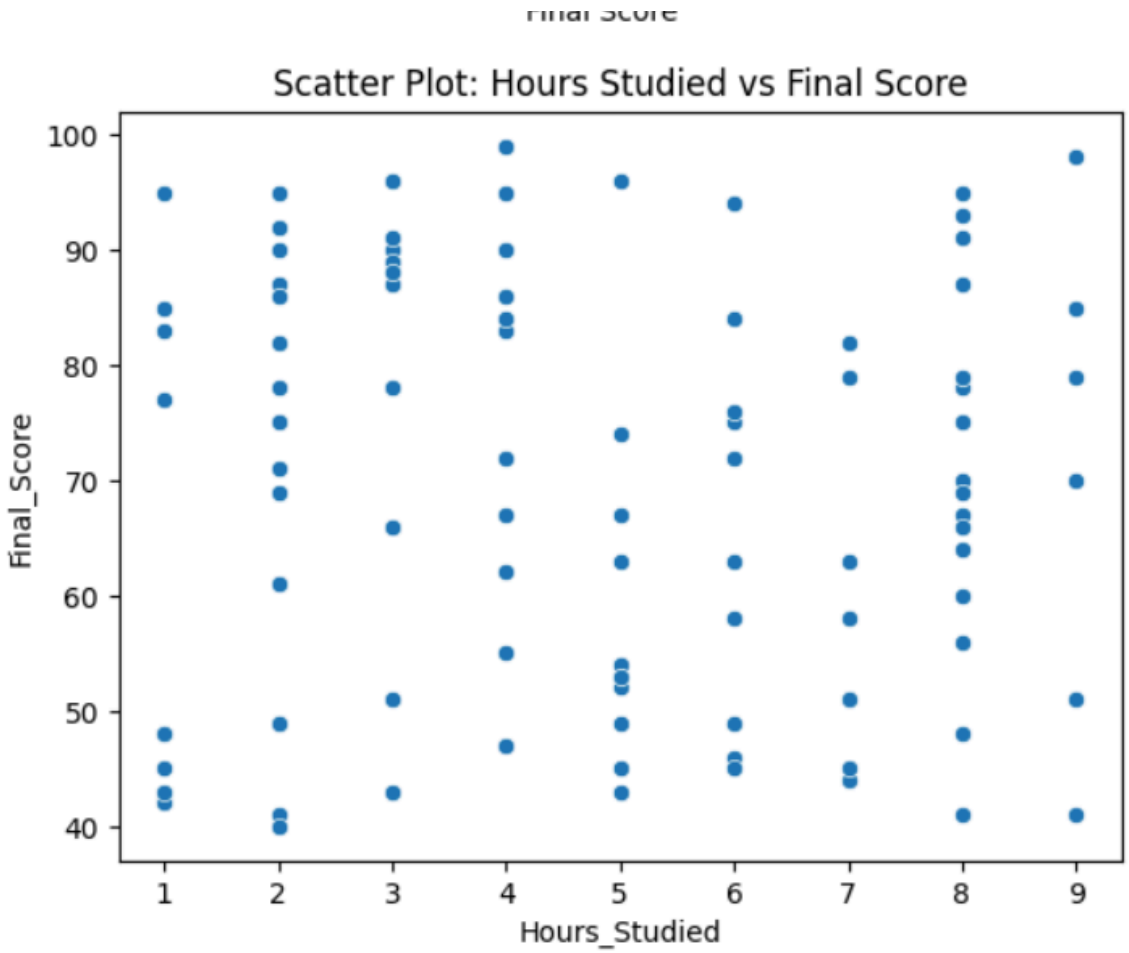
```
# Scatter plot

sns.scatterplot(x="Hours_Studied", y="Final_Score", data=data)

plt.title("Scatter Plot: Hours Studied vs Final Score")

plt.show()



# Heatmap for correlation

plt.figure()

corr = data.corr(numeric_only=True) # Added numeric_only=True for pandas
>= 1.5.0

sns.heatmap(corr, annot=True)

plt.title("Correlation Heatmap")

plt.show()



# Boxplot for categorical analysis

sns.boxplot(x="Performance", y="Final_Score", data=data)

plt.title("Boxplot of Performance vs Final Score")

plt.show()
```

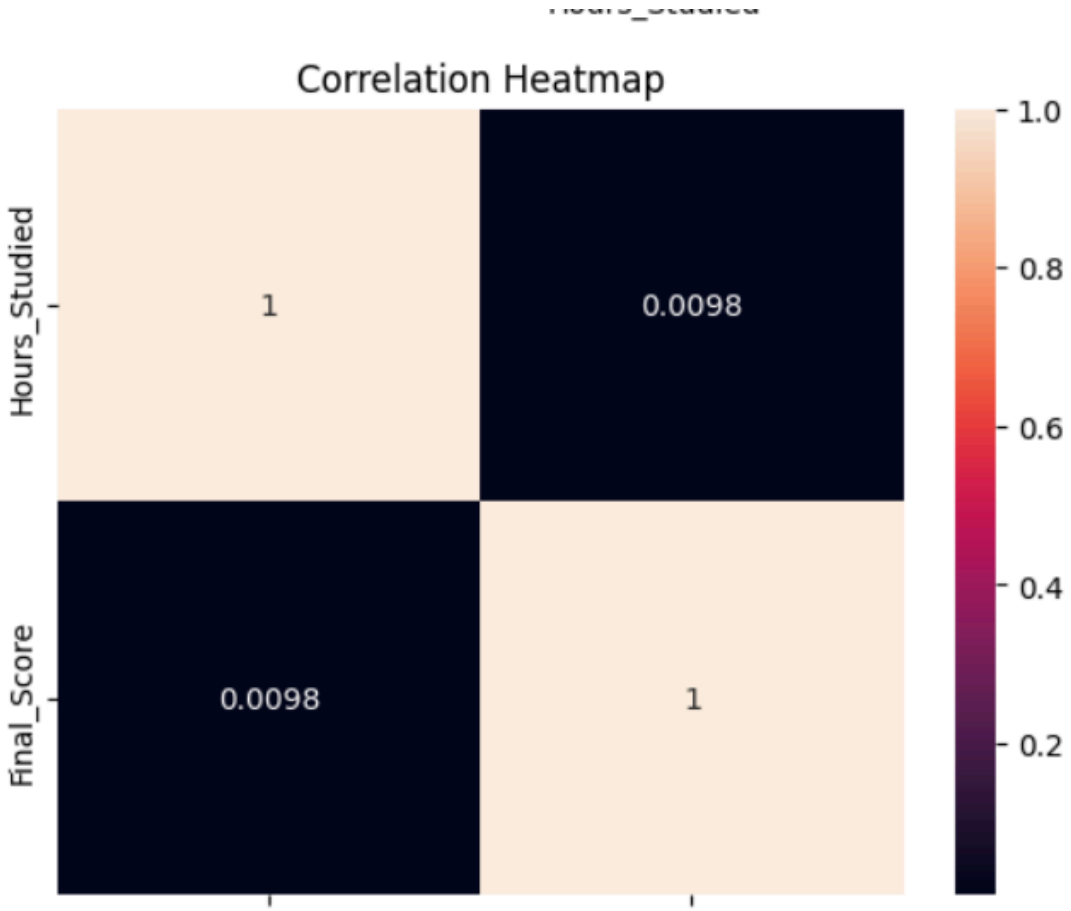
```
Mean: 67.74
Median: 68.0
Standard Deviation: 18.208031195052364
Normalized Final Scores:
 [0.18644068 0.79661017 0.59322034 0.72881356 0.94915254 0.74576271
 0.84745763 0.06779661 0.30508475 0.18644068 0.38983051 0.6440678
 0.50847458 0.15254237 0.86440678 0.05084746 0.76271186 0.01694915
 0.84745763 0.93220339 0.49152542 1.         0.38983051 0.05084746
 0.25423729 0.88135593 0.27118644 0.06779661 0.10169492 0.49152542
 0.93220339 0.89830508 0.08474576 0.45762712 0.79661017 0.91525424
 0.3559322  0.13559322 0.40677966 0.98305085 0.15254237 0.71186441
 0.01694915 0.22033898 0.71186441 0.94915254 0.83050847 0.05084746
 0.6440678  0.03389831 0.33898305 0.15254237 0.57627119 0.18644068
 0.72881356 0.62711864 0.20338983 0.13559322 0.59322034 0.6440678
 0.84745763 0.         0.37288136 0.23728814 0.59322034 0.77966102
 0.01694915 0.13559322 0.08474576 0.45762712 0.79661017 0.30508475
 0.52542373 0.08474576 0.86440678 0.81355932 0.50847458 0.11864407
 0.54237288 0.66101695 0.13559322 0.45762712 0.03389831 0.66101695
 0.61016949 0.08474576 0.15254237 0.74576271 0.38983051 0.44067797
 0.77966102 0.44067797 0.66101695 0.76271186 0.22033898 0.54237288
 0.18644068 0.93220339 0.05084746 0.93220339]
Shape of dataset: (100, 2)
Columns: Index(['Hours_Studied', 'Final_Score'], dtype='object')
Missing values:
 Hours_Studied    0
Final_Score      0
dtype: int64
   Hours_Studied  Final_Score Performance
0              7           51        Good
```

0	7	51	Good
1	2	87	Excellent
2	8	75	Excellent
3	4	83	Excellent
4	3	96	Excellent


Hours Studied vs Final Score
Final Score
Hours Studied
100
90
80
70
60
50
40
1
2
3
4
5
6
7
8
9


Histogram of Final Score
Frequency
Final Score
0
2
4
6
8
10
12
14
16
40
50
60
70
80
90
100


Final Score
Scatter Plot: Hours Studied vs Final Score
100
90
80
70
60
50
40
Final_Score
1
2
3
4
5
6
7
8
9
Hours_Studied


Correlation Heatmap
Hours_Studied
Final_Score
1
0.0098
0.0098
1
1.0
0.8
0.6
0.4
0.2

Tab 2

Tab 3